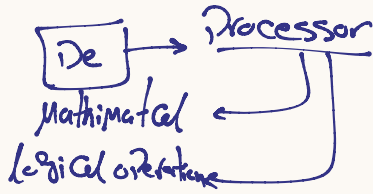
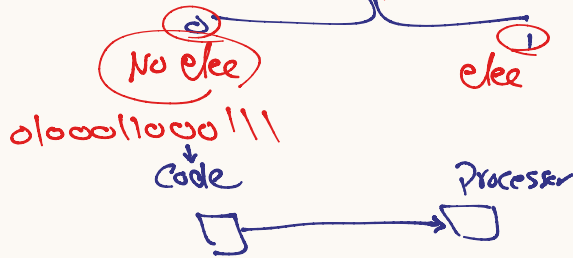


* C_Course

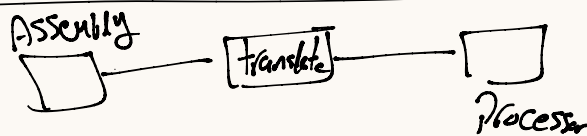
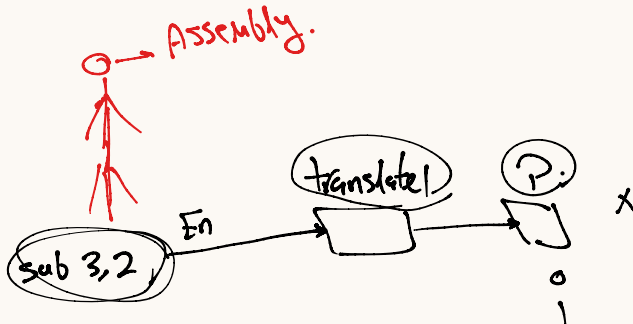
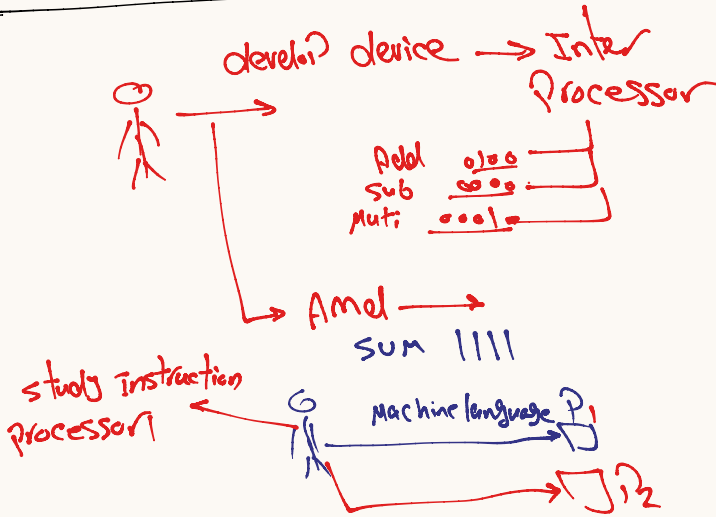
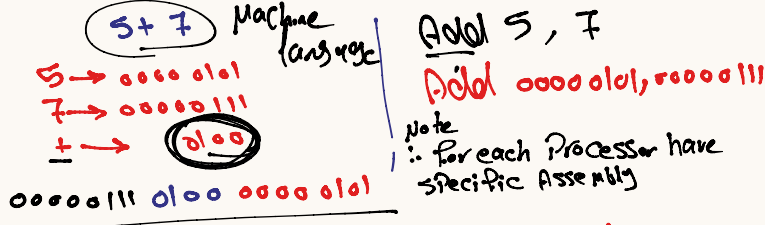
* PL



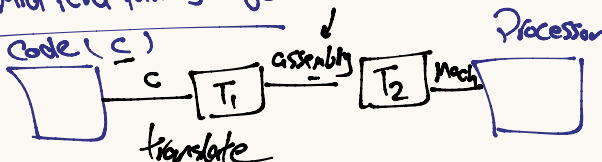
* low level language → * Machine language



* Assembly language.

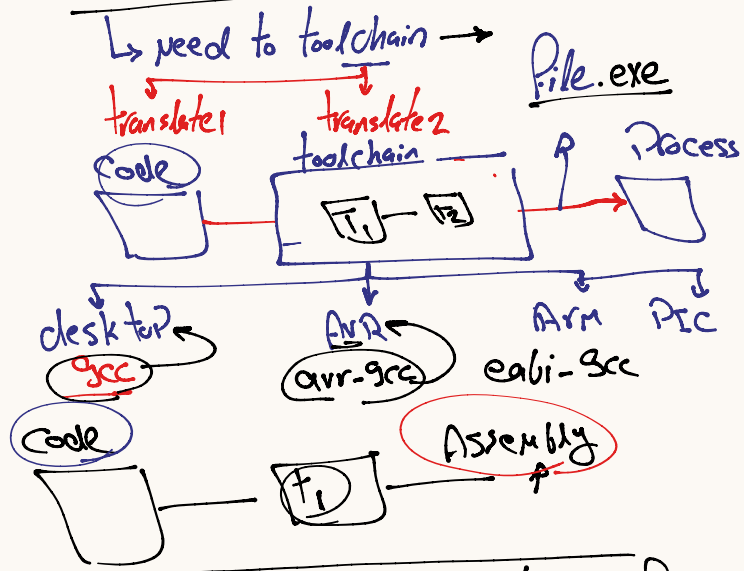


* Mid level language

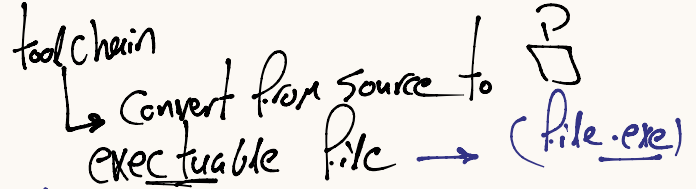


* C language

→ mid level language

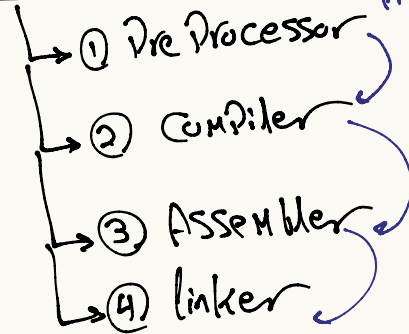


→ need to the executable file.



→ to Run the Program → Run the file.exe

* toolchain Part → Provide the exe file.



Break until at 5:10

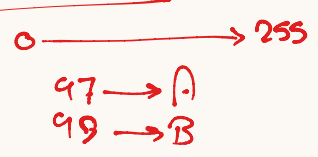
* Printf for variable

```
int x = 30;
printf("x = %d", x);
```

x =

Format specifier
 %d
 ↳ decimal
 %f
 ↳ Floating
 %c
 ↳ characters

to store character



* Input → scanf → stdio.h
 Function

- ① allocate in Ram
- ② scanf

```
ex int Number;
scanf("%d", &Number);
```

location
 Address operator

```
char Id;
scanf("%d", &Id);
char degree;
scanf("%c", &degree);
```

* MISRA-C → guideline to write clean code b/c

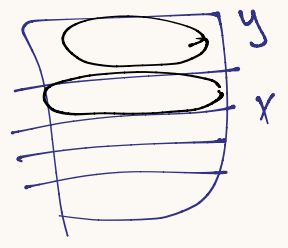
- ↳ naming must be descriptive
 - ↳ not use declaration change to definition b/c
- ```
char No;
char No = 0;
```

\* Casting Concept

Implicit  
 explicit Casting

```
char y = 3;
int x = 5;
```

```
y = x;
```



```
float z;
int No1 = 5;
int No2 = 3;
```

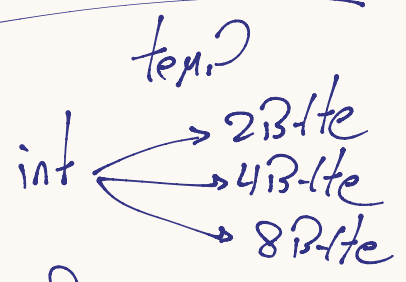
```
z = No1 / No2;
```

```
z = (float) No1 / No2;
```

```
y = No1 + No2
```

issue | Resolve

Some key word



Modifier

↳ Signed Modifier

\* unsigned char temp? → +ve only  
 ↳ 0:255

\* signed char temp? → +ve -ve  
 ↳ 0:127 → +ve  
 ↳ -1:-128 → -ve

(signed/unsigned)

↳ char  
 ↳ int

size Modifier (int/double 2B-ite)

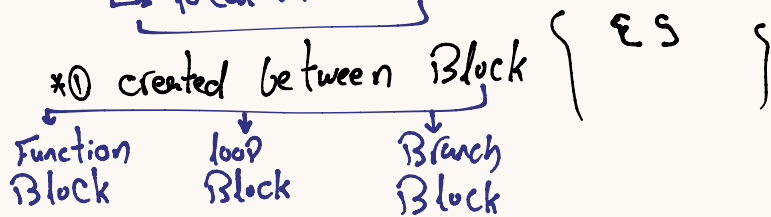
short  
 ↳ int ∴ short int

## Variable and Memories

→ variable will be store in RAM.

### variable life

→ local variable



Finish {  
    char x = 5; → local var  
    printf("x=%d", x);  
}  
→ printf("x=%d", x); → error

\*② local var stored in stack section in RAM

\*③ After Block finish you don't have more Access on this variable  
scope of local var    life time of local var

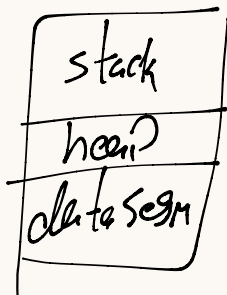
Block scope

Block life time

## Global var

→ variable created outside any Block

→ global var stored in data segment



long  
→ int ∴ long int No  
double ∴ long double

long long → int

→ size operator  
short int No = 5000;

printf(" %d \n", sizeof(No));

ex:

#include <stdio.h>

int main()

char x = 50;  
int x = 30; → issue same name different type

char y = 60;  
char y = 100; → issue same name same type different value

float z;  
float z = 100.3;  
return 0;

scope    global var  
program scope    life time

program scope

program life time

ex:

#include <stdio.h>

→ char x = 30;  
x = 40;

int main()

char x = 70;  
printf("x");  
return 0;



RAM

